Karma Connect

Cleanup Report - what each item is, and what removing it saves

Generated 2026-06-17

These cleanup items cost $0 to keep - none of them make AI calls.

Removing them saves clutter, confusion and risk - not money. The real dollar savings (shown at the end) came from the token + model changes already shipped.

The items, one by one

1. Five generated PDF reports committed to the repo

What it is: Cost / token report PDFs produced by the scripts, accidentally checked into git.
What it does now: Nothing at runtime - they just sit in the repo, add weight, and go out of date as numbers change.
Money saved by removing: $0 (they are files, not API calls).
Real benefit: Cleaner, smaller repo; no stale numbers shipped to anyone who clones it. Keep the files on disk, stop versioning them.

2. setup.bat / setup.command

What it is: "One-click" installers from the old download-a-ZIP distribution model (Windows + macOS).
What it does now: Writes a .env containing REMOVED Supabase variables and NO database / auth / Resend keys - i.e. a broken config. Assumes the old Vite-only app.
Money saved by removing: $0.
Real benefit: Stops a teammate from creating a broken setup. They are actively misleading now.

3. start.sh

What it is: A macOS dev launcher.
What it does now: Looks for a bundled Node in a .tools/ folder that no longer exists. On this Windows PC you use "Start Karma Connect.bat" instead.
Money saved by removing: $0.
Real benefit: Removes a dead, OS-specific launcher.

4. login.sh / push.sh

What it is: macOS helpers that used a bundled GitHub CLI to log in and push to main.
What it does now: Reference a .tools/ gh binary that is gone -> broken. Pushing is done with normal git now.
Money saved by removing: $0.
Real benefit: Removes broken, OS-specific git scripts.

5. SETUP.md

What it is: Setup instructions for the old ZIP / setup.bat flow.
What it does now: Documents the wrong things (localhost:5173, VITE_ANTHROPIC_API_KEY, Supabase). Would mislead a new person.
Money saved by removing: $0.
Real benefit: Either delete, or rewrite for the current Render + Postgres flow so docs are accurate.

6. The "deep" web-search tier (code)

What it is: An unused option in the AI client for 4 web searches per call.
What it does now: Never called - only "light" (1 search) is used. A dead code branch; it never runs.
Money saved by removing: $0 (it is not executed, so it costs nothing even today).
Real benefit: Marginally simpler code. Low priority - safe to leave.

7. Loader caption "Scoring fit..."

What it is: A line in the loading animation shown during analysis.
What it does now: Says the AI is "scoring fit" although the AI no longer does that (the code scorer does). Cosmetic mismatch only.
Money saved by removing: $0.
Real benefit: Accurate on-screen wording.

Where the money actually WAS saved (for context)

Change (already shipped)	Saves / month*
Message variations 10 -> 3	~$120
Hybrid model (Haiku for parse + research)	~$95 (research-heavy)
Lighter company research (3 -> 1 search)	folded into above
Removed unused AI fit scores	~$4
This cleanup report's items	**$0**

* Estimates at 2,000 typical prospects / month on Sonnet. These are the real cost levers; the cleanup items above are about code quality and safety, not spend.

Bottom line

Removing the cleanup items saves $0 per month - but makes the project safer (no broken installer), clearer (accurate docs/UI), and tidier (no artifacts in git). It is worth doing for hygiene, not for cost. All actual savings already came from the token + model work.